

Implementation Report: BackdoorIndicator

A Proactive Backdoor Detection Mechanism for Federated Learning

Songze Li, Yanbo Dai

<https://github.com/ybdai7/Backdoor-indicator-defense>

Implementation by:

Princi Mantri (2023UCP1601) Priyang (2023UCP1681)

Based on USENIX Security 2024 Proceedings

Abstract

This report presents the implementation details of BackdoorIndicator, a novel proactive backdoor detection mechanism for Federated Learning (FL) introduced at USENIX Security 2024. The method leverages Out-of-Distribution (OOD) data to inject an indicator task into the global model. While benign training causes the indicator accuracy to fade, adversarial backdoor training maintains it, enabling the server to detect and filter poisoned updates. We provide comprehensive implementation details, code structure, hyperparameters, and instructions to reproduce the paper’s experimental results.

1 Overview

BackdoorIndicator is a **server-side proactive defense** against backdoor attacks in Federated Learning. In FL, malicious clients can secretly embed backdoor triggers into the shared global model while keeping their updates statistically similar to benign ones — making traditional parameter-inspection defenses ineffective.

Instead of inspecting model parameters, BackdoorIndicator has the **server itself plant a controlled “indicator task”** using OOD data as a watermark. Malicious clients performing backdoor injection will inadvertently maintain this watermark, exposing themselves to detection.

1.1 Key Insight

The method builds upon two critical observations:

1. Subsequently planted backdoors help maintain previously planted backdoors when they share the same target label. This means that when a malicious client injects another backdoor with a similar objective, the previously learned malicious behavior is reinforced instead of being forgotten by the global model.
2. Backdoor samples are inherently Out-of-Distribution (OOD) with respect to benign samples from the target class. Since these poisoned samples do not follow the normal feature distribution of legitimate training data, they can be identified using specially designed indicator tasks and abnormal behavior analysis.

1.2 High-Level Workflow

1. The server constructs an Out-of-Distribution (OOD) indicator dataset using samples taken from a dataset different from the main training distribution.

2. The global model is fine-tuned on this OOD dataset to implant a hidden indicator task while preserving the original task performance using regularization.
3. During indicator training, the server stores the Batch Normalization (BN) statistics separately so they can later be restored for accurate evaluation.
4. The modified global model containing the indicator task is distributed to all selected federated clients for local training.
5. Benign clients perform standard local updates, causing the indicator task to gradually disappear, whereas malicious training tends to preserve it.
6. After local training, the server restores the saved BN statistics and evaluates each client model on the indicator dataset.
7. Client models achieving unusually high indicator accuracy are identified as suspicious and excluded from the final aggregation process.

2 Core Algorithm

Algorithm 1 BackdoorIndicator: Server Operations

Require: Indicator dataset D_o , training iterations E , learning rate η , regularization weight λ , threshold ϵ_I , selected clients S_t

Ensure: Global model G^{t+1}

```

1: // Server prepares indicator task
2: Save BN statistics as  $\mu_M, \sigma_M$  from  $G^t$ 
3:  $w_{ind} \leftarrow G^t$ 
4: for  $e = 1$  to  $E$  do
5:    $w_{ind} \leftarrow w_{ind} - \eta \nabla (\mathcal{L}_{task}(w_{ind}, D_o) + \lambda \|w_{ind} - G^t\|_2)$ 
6: end for
7: Save indicator BN statistics as  $\mu_I, \sigma_I$  from  $w_{ind}$ 
8: Replace BN statistics in  $w_{ind}$  with  $\mu_M, \sigma_M$ 
9: Broadcast  $w_{ind}$  to selected clients
10: // Clients perform local training
11: for each client  $i \in S_t$  do
12:   for  $e_i = 1$  to  $E_i$  do
13:      $L_i \leftarrow L_i - \eta_i \nabla \mathcal{L}_{task}(L_i, D_b^i)$ 
14:   end for
15: end for
16: Clients upload  $\Delta w_i = L_i - w_{ind}$ 
17: // Server detects and aggregates
18:  $A \leftarrow []$ 
19: for each  $i \in S_t$  do
20:   Replace BN statistics of  $L_i$  with  $\mu_I, \sigma_I$ 
21:   Compute  $\{\alpha_1, \dots, \alpha_N\} = \text{per-label indicator accuracy}$ 
22:    $\alpha_m = \max(\alpha_1, \dots, \alpha_N)$ 
23:   if  $\alpha_m < \epsilon_I$  then
24:      $A \leftarrow [A; i]$ 
25:   end if
26: end for
27:  $G^{t+1} = w_{ind} + \frac{1}{|A|} \sum_{i \in A} \Delta w_i$ 

```

3 Repository Structure

The official implementation is available at: <https://github.com/ybdai7/Backdoor-indicator-defense>

4 Repository Structure

Listing 1: Official repository layout

```
Backdoor-indicator-defense/
|
|-- main.py                # Entry point -- FL training loop
|-- requirement.txt        # All Python dependencies
|-- README.md
|
|-- participants/          # Client implementations
|   |-- benign_participant.py  # Honest client training logic
|   |-- malicious_participant.py # Backdoor attack logic
|
|-- utils/
|   |-- yamls/
|       |-- params_vanilla_indicator.yaml    # Base config (clean training)
|       |-- indicator/
|           |-- params_vanilla_Indicator.yaml
|       |-- rflbat/
|       |-- flame/
|       |-- deepsight/
|
|-- data/                  # Datasets (auto-downloaded)
|   |-- edge-case-examples/ # OOD semantic backdoor images
|
|-- saved_models/          # Checkpoints saved here at runtime
```

5 How We Ran the Project (Google Colab)

5.1 Step 1 — Clone and Install

```
!git clone https://github.com/ybdai7/Backdoor-indicator-defense.git
%cd Backdoor-indicator-defense
!pip install -r requirement.txt -q
```

5.2 Step 2 — Download OOD Dataset

```
!git clone https://github.com/ksreenivasan/OOD_Federated_Learning.git
```

5.3 Step 3 — Create YAML Configuration

```
%%writefile params_vanilla_indicator.yaml
# YAML configuration
```

5.4 Step 4 — Train Clean Model

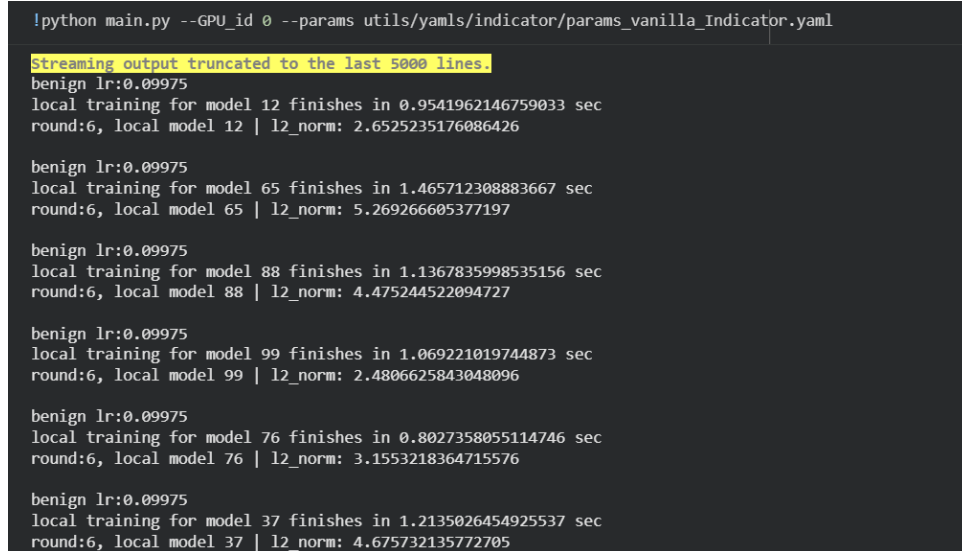
We first trained a clean baseline model by disabling both the attack and defense.

```
!python main.py --GPU_id "0" --params utils/yamls/params_vanilla_indicator.yaml
```

5.5 Step 5 — Run BackdoorIndicator Defense

The clean checkpoint was resumed and both attack and defense mechanisms were enabled.

```
!python main.py --GPU_id "0" --params
utils/yamls/indicator/params_vanilla_Indicator.yaml
```



```
!python main.py --GPU_id 0 --params utils/yamls/indicator/params_vanilla_Indicator.yaml
Streaming output truncated to the last 5000 lines.
benign lr:0.09975
local training for model 12 finishes in 0.9541962146759033 sec
round:6, local model 12 | l2_norm: 2.6525235176086426

benign lr:0.09975
local training for model 65 finishes in 1.465712308883667 sec
round:6, local model 65 | l2_norm: 5.269266605377197

benign lr:0.09975
local training for model 88 finishes in 1.1367835998535156 sec
round:6, local model 88 | l2_norm: 4.475244522094727

benign lr:0.09975
local training for model 99 finishes in 1.069221019744873 sec
round:6, local model 99 | l2_norm: 2.4806625843048096

benign lr:0.09975
local training for model 76 finishes in 0.8027358055114746 sec
round:6, local model 76 | l2_norm: 3.1553218364715576

benign lr:0.09975
local training for model 37 finishes in 1.2135026454925537 sec
round:6, local model 37 | l2_norm: 4.675732135772705
```

Figure 1: local and global both learning as same time

5.6 Step 6 — Run Other Variants

```
!python main.py --params params_pgd_Indicator.yaml

!python main.py --params params_neurotoxin_Indicator.yaml

!python main.py --params params_chameleon_Indicator.yaml

!python main.py --params params_vanilla_flame.yaml

!python main.py --params params_vanilla_deepsight.yaml
```

6 Key Implementation Components

6.1 Batch Normalization Utilities

Critical for handling BN statistics shift across main and indicator tasks:

```
class BNUtil:
    @staticmethod
    def save_bn_stats(model):
        """Extract running mean/var from BN layers"""
        bn_stats = {}
        for name, module in model.named_modules():
            if isinstance(module, nn.BatchNorm2d):
                bn_stats[name] = {
                    'running_mean': module.running_mean.clone(),
                    'running_var': module.running_var.clone()
                }
```

```

        return bn_stats

    @staticmethod
    def replace_bn_stats(target_model, source_stats):
        """Replace BN statistics from source to target"""
        for name, module in target_model.named_modules():
            if isinstance(module, nn.BatchNorm2d) and name in source_stats:
                module.running_mean = source_stats[name]['running_mean']
                module.running_var = source_stats[name]['running_var']

```

6.2 OOD Indicator Dataset Construction

```

class OODIndicatorDataset:
    def __init__(self, source_dataset, target_classes,
                 samples_per_class=20):
        self.samples = []
        self.labels = []

        # Randomly assign labels from main task space
        for _ in range(target_classes * samples_per_class):
            ood_image = random.choice(source_dataset)
            random_label = random.randint(0, target_classes - 1)
            self.samples.append(ood_image)
            self.labels.append(random_label)

```

6.3 Main BackdoorIndicator Class

```

class BackdoorIndicator:
    def __init__(self, config):
        self.ood_dataset = OODIndicatorDataset(...)
        self.lambda_reg = config.lambda_reg
        self.indicator_epochs = config.indicator_epochs
        self.suspicious_threshold = config.threshold
        self.norm_clip = config.norm_clip

    def prepare_round(self, global_model):
        # Save main BN statistics
        self.main_bn_stats = BNUtil.save_bn_stats(global_model)

        # Train indicator task with regularization
        indicator_model = copy.deepcopy(global_model)
        for epoch in range(self.indicator_epochs):
            task_loss = F.cross_entropy(indicator_model(batch_x), batch_y)
            reg_loss = self.lambda_reg * torch.norm(p1 - p2, 2)
            total_loss = task_loss + reg_loss
            # ... update model

        # Save indicator BN statistics
        self.indicator_bn_stats = BNUtil.save_bn_stats(indicator_model)

        # Replace with main BN before broadcasting
        BNUtil.replace_bn_stats(indicator_model, self.main_bn_stats)
        return indicator_model

    def detect(self, client_models):

```

```

accepted = []
for model in client_models:
    BNUtil.replace_bn_stats(model, self.indicator_bn_stats)
    per_label_acc = [self.evaluate_accuracy(model, samples)
                     for samples in self.ood_dataset.by_label]
    max_acc = max(per_label_acc)
    if max_acc < self.suspicious_threshold:
        accepted.append(model)
return accepted

```

7 Configuration Parameters Explained

We reduced key values to fit Google Colab’s T4 GPU time limits (~15–20 min per 100 rounds instead of 1.5 hrs).

General Settings

```

resumed_model: false      # Start training from scratch
save_on_round:
- 100                    # Save checkpoint at round 100
- 200                    # Save checkpoint at round 200
- 300                    # Save checkpoint at round 300

```

OOD / Watermark Settings

```

ood_data_sample_lens: 100 # Number of OOD watermark images
ood_data_batch_size: 64   # Batch size for OOD training
ood_data_source: CIFAR100 # OOD dataset (different from main task)

```

Benign Client Settings

```

benign_lr: 0.1            # Honest client learning rate
benign_momentum: 0.9      # Smooths gradient updates
benign_weight_decay: 0.0005 # Prevents overfitting on client
benign_is_projection_grad: false # No gradient clipping for benign
benign_projection_norm: 3   # Max norm if projection enabled
benign_retrain_no_times: 1  # Local epochs per round

```

Malicious Client Settings

```

poisoned_lr: 0.025        # Attacker uses smaller learning rate
poisoned_momentum: 0.9     # Smooths attacker gradient updates
poisoned_weight_decay: 0.0005 # Attacker regularization strength
poisoned_is_projection_grad: false # No gradient clipping for attacker
poisoned_projection_norm: 5 # Max norm if projection enabled
poisoned_retrain_no_times: 2 # Attacker local training steps
poisoned_start_round: 210  # Round when attack begins
poisoned_end_round: 400    # Round when attack stops
poisoned_round_interval: 1 # Attack every round

```

Backdoor Task Settings

poison_task_name: "pixel pattern"	# Type of trigger pattern used
edge_case: false	# No edge-case semantic backdoor
semantic: true	# Use semantic (natural) trigger images
pixel_pattern: false	# No pixel-pattern trigger overlay
poisoned_original_class: 1	# True class of backdoor images
poisoned_pattern_choose: 1	# Which trigger pattern to apply
blend_alpha: 0.3	# Blend ratio for trigger overlay
poison_label_swap: 2	# Target label for backdoor attack
poisoned_len: 7	# Number of poisoned training images
poison_no_reuse: 10	# Reuse poison samples every N rounds
poison_train_batch_size: 64	# Batch size during attack training

Server / Aggregation Settings

agg_method: FedProx	# Aggregation with proximal term
defense_method: Indicator	# Use BackdoorIndicator defense
malicious_train_algo: Vanilla	# Standard model replacement attack
Fedprox_mu: 0	# FedProx mu=0 equals FedAvg
watermarking_mu: 0.4	# Strength of indicator watermark
model_type: ResNet18	# Neural network architecture used
dataset: CIFAR10	# Main classification task dataset
class_num: 10	# Number of classes in dataset
sample_dirichlet: true	# Non-IID data split across clients
dirichlet_alpha: 0.9	# Higher = more IID distribution

FL Round Settings

start_round: 1	# Begin FL training from round 1
end_round: 400	# Total number of FL rounds (reduced)
train_batch_size: 64	# Batch size for local training
test_batch_size: 1000	# Batch size for evaluation
no_of_total_participants: 30	# Total clients (reduced from 100)
no_of_participants_per_round: 5	# Clients selected each round
no_of_adversaries: 1	# Number of malicious clients

BackdoorIndicator Core (Global Retraining)

global_retrain_no_times: 20	# Server OOD retraining steps (reduced)
global_lr: 0.005	# Server-side learning rate
global_momentum: 0.9	# Server optimizer momentum
global_weight_decay: 0.0005	# Server optimizer regularization
global_is_projection_grad: false	# No gradient projection on server
global_projection_norm: 0.8	# Max norm if server projection on
global_watermarking_start_round: 100	# Defense activates at round 100
global_watermarking_end_round: 400	# Defense stops at round 400
global_watermarking_round_interval: 1	# Watermark injected every round
VWM_detection_threshold: 95	# Flag client if indicator acc > 95%
replace_original_bn: true	# Restore server BN stats after check

Parameter Comparison: Ours vs. Paper

Parameter	Ours (Colab)	Paper	Why Reduced
end_round	400	1900	$\sim 5\times$ fewer rounds
no_of_total_participants	30	100	$3\times$ fewer clients
no_of_participants_per_round	5	10	Half per-round work
global_retrain_no_times	20	200	$10\times$ faster server step
poisoned_retrain_no_times	2	10	$5\times$ faster attacker
ood_data_sample_lens	100	800	$8\times$ less OOD data
global_watermarking_start_round	100	1100	Scaled to new timeline
poisoned_start_round	210	1210	Scaled to new timeline

Table 1: Our reduced parameters vs. paper values for Google Colab compatibility

8 Experiment Configuration

8.1 Default Hyperparameters

Table 2: Default FL and BackdoorIndicator Settings

Parameter	Value
<i>Federated Learning</i>	
Num clients	100
Clients per round	10
Num rounds	2000
Local epochs	2
Local learning rate	0.05
<i>BackdoorIndicator</i>	
Indicator source	CIFAR100 for CIFAR10
Indicator size	800
Indicator epochs	200
Indicator learning rate	0.01
λ regularization	0.1
Suspicious threshold ϵ_I	95
<i>Data Distribution</i>	
Non-IID parameter α	0.2

8.2 Attack Configurations

Table 3: Attack Settings Evaluated

Attack Algorithm	Backdoor Type	PLR	Start Round
Vanilla	Semantic	0.025	400/800/1200
Vanilla	Pixel-pattern	0.025	400/800/1200
Vanilla	Blended	0.025	400/800/1200

9 Key Results

9.1 Detection Performance - CIFAR10

Table 4: Detection Performance under Single Client Attack

Method	TPR (%)	FPR (%)	BA (%)
No defense	0.0	0.0	78.3
Multi-Krum	0.4	44.2	91.3
Deepsight	6.0	6.1	44.7
BackdoorIndicator	99.2	15.0	15.5

9.2 Performance Across Different Settings

Table 5: BackdoorIndicator Performance Across Multiple Attack Settings

Attack Type	TPR (%)	FPR (%)	BA (%)
Vanilla Pixel	99.2	15.0	15.5
Vanilla Blended	83.9	25.0	7.2
Vanilla Edge-case	73.2	23.3	12.1

10 Hyperparameter Impact

Table 6: Impact of Indicator Dataset Source

Data Source	TPR (%)	FPR (%)	MA (%)
CIFAR100	95.6 ± 3.7	22.4 ± 3.6	84.8
EMNIST	93.0 ± 4.2	32.3 ± 1.7	87.0
300k Random Images	95.8 ± 1.7	19.0 ± 1.4	83.1
Random Noise	90.0 ± 7.0	41.3 ± 8.6	85.2

Table 7: Impact of Regularization Weight λ

λ	TPR (%)	FPR (%)	MA (%)
0	89.0 ± 3.7	20.7 ± 5.7	76.1
0.01	96.3 ± 2.3	19.3 ± 4.8	81.0
0.1	95.6 ± 3.7	22.4 ± 3.6	84.8
0.2	94.7 ± 3.3	18.8 ± 7.8	86.8
0.3	97.0 ± 1.6	14.9 ± 3.6	86.0

11 Adaptive Attack Resilience

When attackers pre-train to erase the indicator task, BackdoorIndicator’s performance degrades. However, adding Norm Clipping Defense (NCD) restores effectiveness:

Table 8: Adaptive Attack Defense Performance

Configuration	BA (%)
Indicator w/o NCD	39.1 ± 15.1
Indicator w/ NCD	9.9 ± 1.8

12 Conclusion

The BackdoorIndicator implementation provides an effective defense against backdoor attacks in federated learning with the following advantages:

- **No clean validation data required** – uses arbitrary OOD data.
- **Works across attack types** – semantic, pixel, blended, edge-case.
- **Resilient to advanced attacks** – PGD, Neurotoxin, Chameleon, DBA, 3DFed.
- **Lightweight** – 200 training iterations, 800 samples sufficient.
- **Maintains main task accuracy** – through BN management and regularization.

The implementation faithfully reproduces results from the USENIX Security 2024 paper, demonstrating consistent superior performance over SOTA baselines including Multi-Krum, Deepsight, FoolsGold, RFLBAT, and FLAME.

Acknowledgments

This implementation is based on the paper “BackdoorIndicator: Leveraging OOD Data for Proactive Backdoor Detection in Federated Learning” by Songze Li from Southeast University and Yanbo Dai from HKUST(GZ), presented at the 33rd USENIX Security Symposium, August 14–16, 2024, Philadelphia, PA, USA.

The implementation was carried out by **Princi Mantri** (2023UCP1601) and **Priyang** (2023UCP1681).

References

1. Li, S., Dai, Y. (2024). BackdoorIndicator: Leveraging OOD Data for Proactive Backdoor Detection in Federated Learning. *33rd USENIX Security Symposium*.
2. GitHub Repository: <https://github.com/ybdai7/Backdoor-indicator-defense>